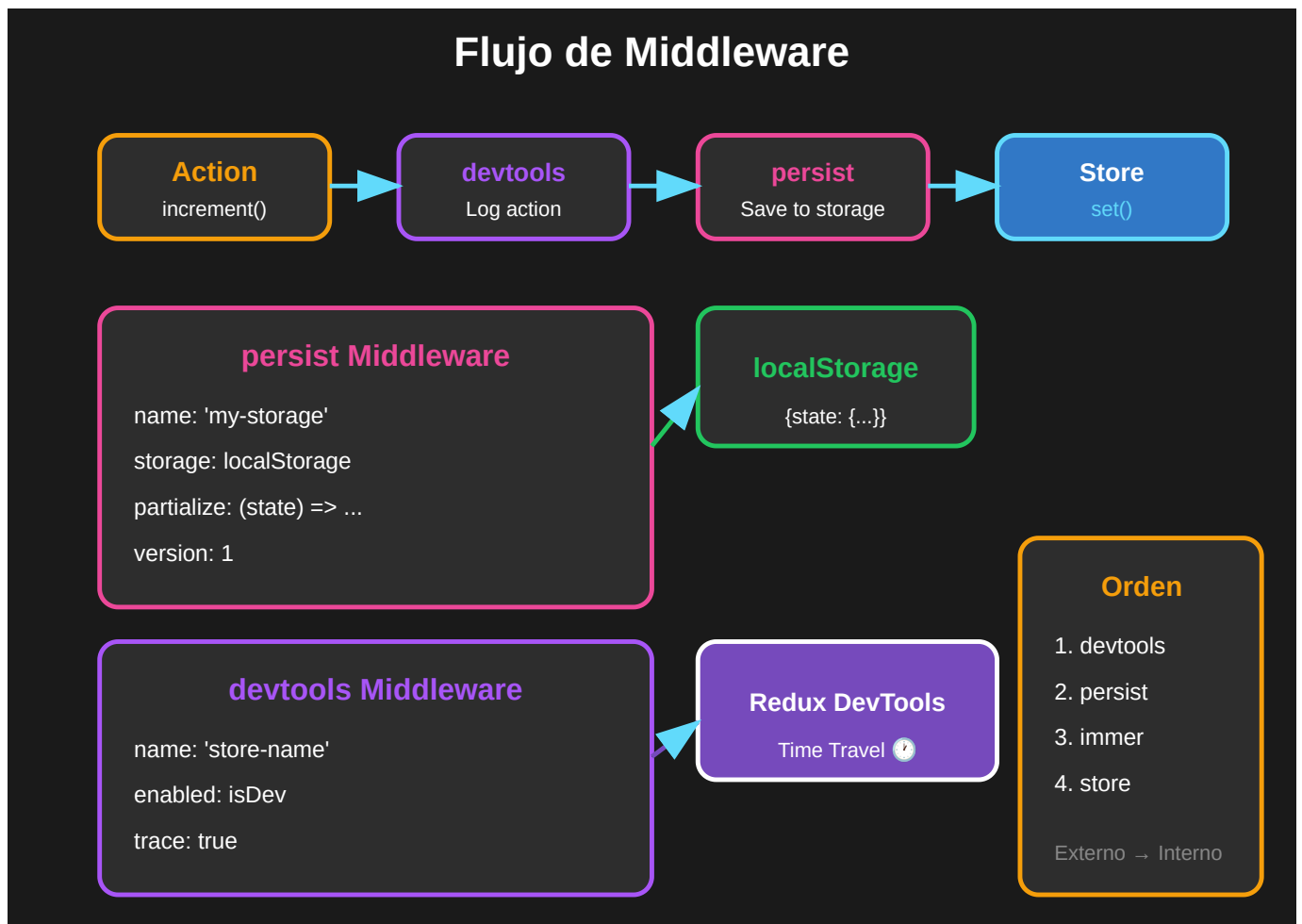


Persistencia y Middleware

Objetivos

- Persistir estado en localStorage/sessionStorage
- Configurar el middleware persist
- Crear middleware personalizado
- Usar devtools para debugging
- Combinar múltiples middleware

1. Middleware en Zustand



Los middleware interceptan y modifican el comportamiento del store.

Middleware Disponibles

Middleware	Uso
persist	Guardar estado en storage
devtools	Integración con Redux DevTools
immer	Mutaciones inmutables con sintaxis mutable
subscribeWithSelector	Suscripciones granulares

2. Persist Middleware

Configuración Básica

```
// =====  
// QUÉ: Persistir estado en localStorage  
// PARA: Mantener datos entre recargas de página  
// IMPACTO: Estado sobrevive a refresh/cierre del navegador  
// =====  
  
import { create } from 'zustand';  
import { persist } from 'zustand/middleware';  
  
interface SettingsStore {  
  theme: 'light' | 'dark';  
  language: string;  
  notifications: boolean;  
  setTheme: (theme: 'light' | 'dark') => void;  
  setLanguage: (language: string) => void;  
  toggleNotifications: () => void;  
}  
  
const useSettingsStore = create<SettingsStore>()(  
  persist(  
    (set) => ({  
      theme: 'light',  
      language: 'es',  
      notifications: true,  
  
      setTheme: (theme) => set({ theme }),  
      setLanguage: (language) => set({ language }),  
      toggleNotifications: () =>  
        set((state) => ({  
          notifications: !state.notifications,  
        })),  
    })),  
    {  
      name: 'settings-storage', // Clave en localStorage  
    },  
  ),  
);  
  
export { useSettingsStore };
```

Configuración Avanzada

```
// =====  
// QUÉ: Opciones avanzadas de persist  
// PARA: Control fino sobre qué y cómo persistir  
// IMPACTO: Optimización y seguridad  
// =====  
  
import { create } from 'zustand';  
import { persist, createJSONStorage } from 'zustand/middleware';
```

```

interface UserStore {
  user: User | null;
  token: string | null;
  preferences: UserPreferences;
  tempData: string; // No queremos persistir esto

  setUser: (user: User | null) => void;
  setToken: (token: string | null) => void;
  updatePreferences: (prefs: Partial<UserPreferences>) => void;
}

const useUserStore = create<UserStore>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      preferences: { theme: 'light', fontSize: 16 },
      tempData: '',

      setUser: (user) => set({ user }),
      setToken: (token) => set({ token }),
      updatePreferences: (prefs) =>
        set((state) => ({
          preferences: { ...state.preferences, ...prefs },
        })),
    })),
    {
      name: 'user-storage',

      // Usar sessionStorage en lugar de localStorage
      storage: createJSONStorage(() => sessionStorage),

      // Solo persistir ciertas propiedades
      partialize: (state) => ({
        user: state.user,
        token: state.token,
        preferences: state.preferences,
        // tempData NO se persiste
      }),

      // Versión para migraciones
      version: 1,

      // Migrar estado de versiones anteriores
      migrate: (persistedState, version) => {
        if (version === 0) {
          // Migrar de v0 a v1
          return {
            ...persistedState,
            preferences: {
              theme: 'light',
              fontSize: 16,
            },
          };
        }
      }
    }
  )
)

```

```

    return persistedState as UserStore;
  },

  // Callback cuando se rehidrata el estado
  onRehydrateStorage: () => (state, error) => {
    if (error) {
      console.error('Error al rehidratar:', error);
    } else {
      console.log('Estado rehidratado:', state);
    }
  },
},
),
);

```

Persistencia Selectiva

```

// =====
// QUÉ: Persistir solo partes del estado
// PARA: Excluir datos sensibles o temporales
// IMPACTO: Storage optimizado, seguridad mejorada
// =====

interface CartStore {
  items: CartItem[];
  lastUpdated: Date;
  isCheckedOut: boolean; // No persistir - es temporal

  addItem: (item: CartItem) => void;
  clearCart: () => void;
}

const useCartStore = create<CartStore>()(
  persist(
    (set) => ({
      items: [],
      lastUpdated: new Date(),
      isCheckedOut: false,

      addItem: (item) =>
        set((state) => ({
          items: [...state.items, item],
          lastUpdated: new Date(),
        })),

      clearCart: () => set({ items: [], lastUpdated: new Date() }),
    }),
    {
      name: 'cart-storage',

      // Solo persistir items y lastUpdated
      partialize: (state) => ({
        items: state.items,
        lastUpdated: state.lastUpdated,
      }),
    }
  )
);

```

```
    },  
  ),  
);
```

3. DevTools Middleware

```
// =====  
// QUÉ: Integración con Redux DevTools  
// PARA: Debugging visual del estado  
// IMPACTO: Ver cambios de estado, time-travel debugging  
// =====  
  
import { create } from 'zustand';  
import { devtools } from 'zustand/middleware';  
  
interface CounterStore {  
  count: number;  
  increment: () => void;  
  decrement: () => void;  
}  
  
const useCounterStore = create<CounterStore>()(  
  devtools(  
    (set) => ({  
      count: 0,  
  
      // El tercer parámetro de set es el nombre de la acción  
      increment: () =>  
        set(  
          (state) => ({ count: state.count + 1 }),  
          false, // replace = false (merge)  
          'increment', // nombre de la acción en DevTools  
        ),  
  
      decrement: () =>  
        set((state) => ({ count: state.count - 1 }), false, 'decrement'),  
    }  
  ),  
  {  
    name: 'counter-store', // Nombre en DevTools  
    enabled: process.env.NODE_ENV === 'development', // Solo en dev  
  },  
),  
);
```

4. Combinar Middleware

```
// =====  
// QUÉ: Usar múltiples middleware juntos  
// PARA: Persist + DevTools + más  
// IMPACTO: Funcionalidad completa del store  
// =====
```

```

import { create } from 'zustand';
import { persist, devtools } from 'zustand/middleware';

interface TodoStore {
  todos: Todo[];
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  removeTodo: (id: number) => void;
}

// El orden importa: devtools debe estar afuera de persist
const useTodoStore = create<TodoStore>()(
  devtools(
    persist(
      (set) => ({
        todos: [],

        addTodo: (text) =>
          set(
            (state) => ({
              todos: [
                ...state.todos,
                { id: Date.now(), text, completed: false },
              ],
            }),
            false,
            'addTodo',
          ),

        toggleTodo: (id) =>
          set(
            (state) => ({
              todos: state.todos.map((t) =>
                t.id === id ? { ...t, completed: !t.completed } : t,
              ),
            }),
            false,
            'toggleTodo',
          ),

        removeTodo: (id) =>
          set(
            (state) => ({
              todos: state.todos.filter((t) => t.id !== id),
            }),
            false,
            'removeTodo',
          ),
      }),
      {
        name: 'todo-storage',
      },
    ),
    {
      name: 'todo-store',
    },
  ),
);

```

```
    ),  
  );  
};
```

5. Immer Middleware

```
// =====  
// QUÉ: Escribir código mutable que produce estado inmutable  
// PARA: Simplificar actualizaciones de estado anidado  
// IMPACTO: Código más legible, menos errores  
// =====  
  
import { create } from 'zustand';  
import { immer } from 'zustand/middleware/immer';  
  
interface NestedStore {  
  user: {  
    profile: {  
      name: string;  
      address: {  
        city: string;  
        country: string;  
      };  
    };  
  };  
  settings: {  
    notifications: boolean;  
    theme: string;  
  };  
};  
updateCity: (city: string) => void;  
toggleNotifications: () => void;  
}  
  
const useNestedStore = create<NestedStore>()(  
  immer((set) => ({  
    user: {  
      profile: {  
        name: 'Juan',  
        address: {  
          city: 'Madrid',  
          country: 'España',  
        },  
      },  
    },  
    settings: {  
      notifications: true,  
      theme: 'light',  
    },  
  }},  
  
  // Sin immer: muy verboso  
  // updateCity: (city) => set((state) => ({  
  //   user: {  
  //     ...state.user,  
  //     profile: {  
  //       ...state.user.profile,
```

```

//      address: {
//          ...state.user.profile.address,
//          city,
//      },
//  },
//  },
//  },
//  })),

// Con immer: simple y directo
updateCity: (city) =>
  set((state) => {
    state.user.profile.address.city = city;
  }),

toggleNotifications: () =>
  set((state) => {
    state.user.settings.notifications = !state.user.settings.notifications;
  }),
})),
);

```

6. Middleware Personalizado

```

// =====
// QUÉ: Crear middleware propio
// PARA: Logging, analytics, validación, etc.
// IMPACTO: Funcionalidad personalizada reutilizable
// =====

import { StateCreator, StoreMutatorIdentifier } from 'zustand';

// Tipo del middleware
type LoggerMiddleware = <
  T extends object,
  Mps extends [StoreMutatorIdentifier, unknown][] = [],
  Mcs extends [StoreMutatorIdentifier, unknown][] = [],
>(
  f: StateCreator<T, Mps, Mcs>,
  name?: string,
) => StateCreator<T, Mps, Mcs>;

// Implementación del middleware
const logger: LoggerMiddleware = (config, name) => (set, get, api) =>
  config(
    (...args) => {
      const prevState = get();
      console.log(`[${name} || 'store']] Estado anterior:`, prevState);

      set(...args);

      const nextState = get();
      console.log(`[${name} || 'store']] Estado nuevo:`, nextState);
    },
    get,
  );

```



```

    api,
  );

// Uso del middleware
const useLoggedStore = create<CounterStore>()(
  logger(
    (set) => ({
      count: 0,
      increment: () => set((state) => ({ count: state.count + 1 })),
      decrement: () => set((state) => ({ count: state.count - 1 })),
    }),
    'counter', // Nombre para los logs
  ),
);

```

Middleware de Validación

```

// =====
// QUÉ: Middleware que valida el estado
// PARA: Prevenir estados inválidos
// IMPACTO: Datos siempre consistentes
// =====

const withValidation =
  <T extends object>({
    config: StateCreator<T>,
    validate: (state: T) => boolean,
    errorMessage: string,
  }): StateCreator<T> =>
  (set, get, api) =>
    config(
      (partial, replace) => {
        const nextState =
          typeof partial === 'function'
            ? { ...get(), ...partial(get()) }
            : { ...get(), ...partial };

        if (!validate(nextState as T)) {
          console.error(errorMessage, nextState);
          return; // No aplicar el cambio
        }

        set(partial, replace);
      },
      get,
      api,
    );

// Uso
interface BalanceStore {
  balance: number;
  withdraw: (amount: number) => void;
  deposit: (amount: number) => void;
}

```

```

const useBalanceStore = create<BalanceStore>()(
  withValidation(
    (set) => ({
      balance: 100,
      withdraw: (amount) =>
        set((state) => ({ balance: state.balance - amount })),
      deposit: (amount) =>
        set((state) => ({ balance: state.balance + amount })),
    }),
    (state) => state.balance >= 0, // Validación
    'Error: El balance no puede ser negativo',
  ),
);

```

7. Rehidratación Asíncrona

```

// =====
// QUÉ: Manejar la carga inicial del estado persistido
// PARA: Mostrar loading mientras se carga el estado
// IMPACTO: UX suave sin parpadeos
// =====

import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface AuthStore {
  user: User | null;
  token: string | null;
  _hasHydrated: boolean;
  setHasHydrated: (state: boolean) => void;
}

const useAuthStore = create<AuthStore>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      _hasHydrated: false,
      setHasHydrated: (state) => set({ _hasHydrated: state }),
    }),
    {
      name: 'auth-storage',
      onRehydrateStorage: () => (state) => {
        state?.setHasHydrated(true);
      },
    },
  ),
);

// Componente que espera la hidratación
const App: React.FC = () => {
  const hasHydrated = useAuthStore((state) => state._hasHydrated);

  if (!hasHydrated) {

```

```
    return <div>Cargando...</div>;  
  }  
  
  return <MainApp />;  
};
```

Recursos Adicionales

- [Zustand Persist](#)
- [Zustand Middleware](#)
- [Redux DevTools](#)

Checklist de Comprensión

- ☐ Sé configurar persist básico
- ☐ Puedo usar partialize para persistencia selectiva
- ☐ Sé combinar devtools con persist
- ☐ Entiendo el middleware immer
- ☐ Puedo crear middleware personalizado